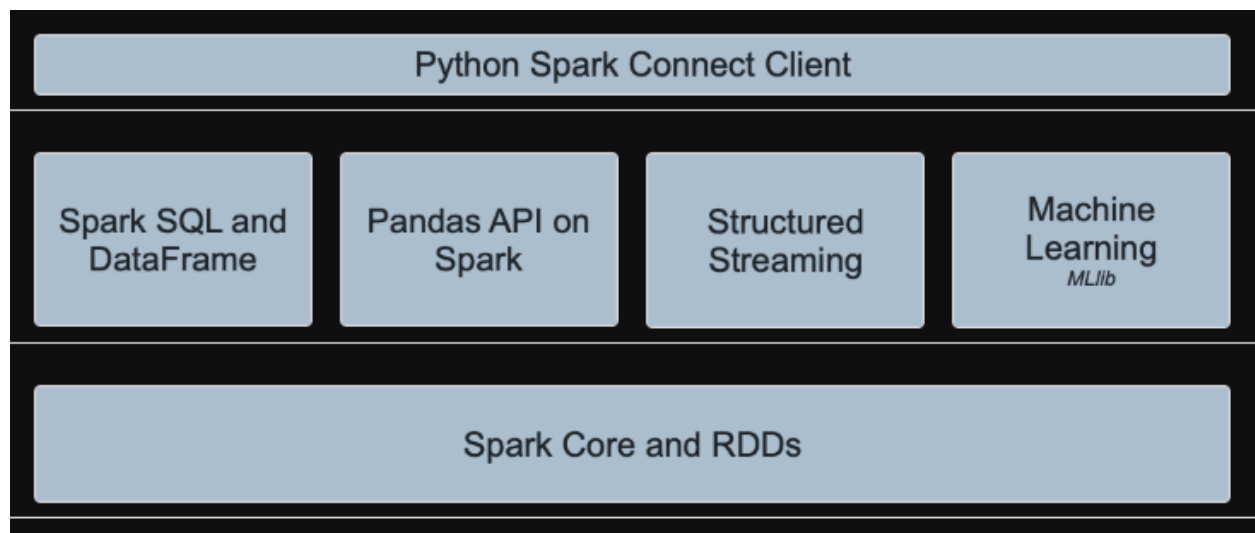


PySpark Overview

PySpark is the **Python API** for **Apache Spark**. It enables you to perform real-time, large-scale data processing in a distributed environment using Python. It also provides a PySpark shell for interactively analyzing your data.

PySpark combines Python's learnability and ease of use with the power of Apache Spark to enable processing and analysis of data at any size for everyone familiar with Python.

PySpark supports all of Spark's features such as **Spark SQL, DataFrames, Structured Streaming, Machine Learning (MLlib), Pipelines and Spark Core.**



Core Abstractions

a) RDD (Resilient Distributed Dataset)

- Low-level distributed collection
- Immutable, fault-tolerant
- Rarely used in modern workflows

```
rdd = spark.sparkContext.parallelize([1,2,3,4])
print(rdd.map(lambda x: x * 2).collect())
```

b) DataFrame (Most Important)

- Distributed table (like SQL table)
- Optimized via **Catalyst Optimizer**
- Preferred API

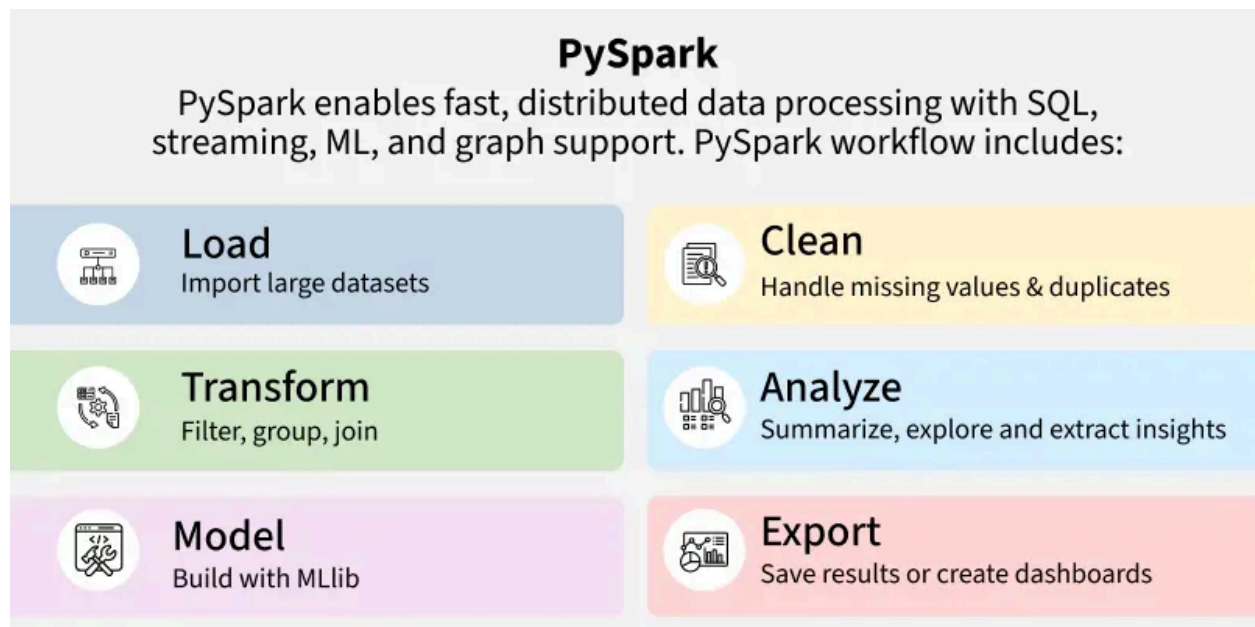
```
df = spark.read.csv("data.csv", header=True, inferSchema=True)
```

df.show()

c) Dataset

- Typed version of DataFrame (mainly in Scala, not used in PySpark)

Key Features



Feature	Description
Lazy Evaluation	Execution happens only when needed
Distributed Processing	Data split across nodes
Fault Tolerance	Recovers from node failures
In-Memory Computation	Faster than disk-based systems
Integration	Works with Hadoop, Hive, Kafka

Basic Workflow

Step 1: Initialize Spark Session

```
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .appName("Example") \
```

```
.getOrCreate()
```

Step 2: Load Data

```
df = spark.read.json("data.json")
```

Step 3: Transform Data

```
df_filtered = df.filter(df.age > 25)  
df_selected = df_filtered.select("name", "age")
```

Step 4: Action (Triggers Execution)

```
df_selected.show()
```

Transformations vs Actions

Type	Examples	Description
Transformations	filter(), select(), groupBy()	Lazy
Actions	show(), collect(), count()	Execute computation

What is PySpark Used For?

PySpark lets you use Python to process and analyze huge datasets that can't fit on one computer. It runs across many machines, making big data tasks faster and easier. You can use PySpark to:

- Perform batch and real-time processing on large datasets.
- Execute SQL queries on distributed data.
- Run scalable machine learning models.
- Stream real-time data from sources like Kafka or TCP sockets.
- Process graph data using GraphFrames.

When to Use PySpark

Use PySpark when:

- Dataset is **large (GBs–TBs)**
- Need **parallel processing**
- Working with **ETL pipelines or big data systems**

ADVANTAGES OF PYSPARK

1. Massive Scalability

- Handles **GB** → **TB** → **PB scale data**
- Distributes computation across multiple nodes (cluster computing)
- Horizontally scalable (add more machines -> more performance)

Implication: Suitable for real-world production data pipelines, not just academic datasets.

2. High Performance (In-Memory Processing)

- Uses **RAM instead of disk** (unlike traditional MapReduce)
- Reduces I/O bottlenecks significantly

Result: Up to **10–100× faster** for iterative workloads (e.g., ML, analytics)

3. Python Ecosystem Integration

- Native Python API -> easy adoption
- Works with:
 - NumPy
 - Pandas (via .toPandas())
 - Matplotlib
 - Scikit-learn

Advantage: You don't need to switch to Scala/Java to use Spark.

4. Unified Engine (Multiple Workloads)

Supports multiple paradigms in one system:

Module	Use Case
Spark SQL	Structured data queries
Spark Streaming	Real-time data
MLlib	Machine learning
GraphX	Graph processing

Benefit: No need to maintain multiple tools.

5. Fault Tolerance

- Uses **lineage (DAG)** to recompute lost data
- Automatically recovers from node failures

Practical impact: Reliable for distributed systems where failures are common.

6. Lazy Evaluation (Optimized Execution)

- Operations are not executed immediately
- Spark builds a **logical plan (DAG)** and optimizes it

Result:

- Reduced redundant computation
- Better execution planning via Catalyst Optimizer

7. Ease of Use with DataFrames

- Similar to SQL tables
- Cleaner and more intuitive than RDDs

```
df.filter(df.age > 25).select("name").show()
```

Advantage: Faster development + fewer bugs.

8. Strong Big Data Ecosystem Support

Integrates with:

- Hadoop (HDFS)
- Apache Hive
- Apache Kafka
- Cloud platforms (AWS, Azure, GCP)

Use case: End-to-end data pipelines.

9. Real-Time + Batch Processing

- Supports both:
 - Batch jobs (ETL)
 - Streaming (real-time analytics)

Example: Fraud detection systems, live dashboards.

10. Cost Efficiency (at Scale)

- Efficient resource utilization
- Can run on commodity hardware clusters

Trade-off:

Higher setup complexity vs long-term scalability benefits.

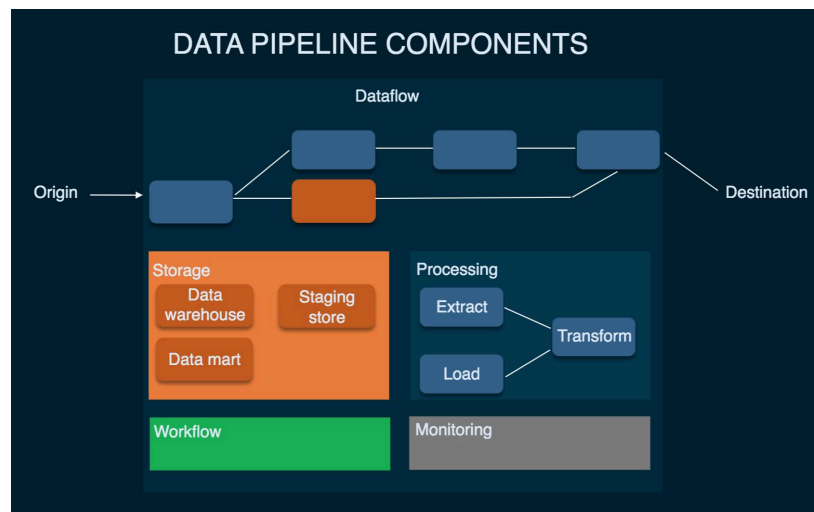
USE CASES OF PYSPARK

1. Data Processing and ETL

Data processing and ETL (extract, transform, load) are critical components in data engineering workflows. Organizations need to extract data from various sources, transform it into a suitable format, and load it into a data warehouse or data lake for analysis.

How Spark can help:

Spark's ability to handle large-scale data processing makes it ideal for ETL tasks. Its in-memory computation significantly speeds up the transformation processes, while its integration with various data sources simplifies data extraction and loading.

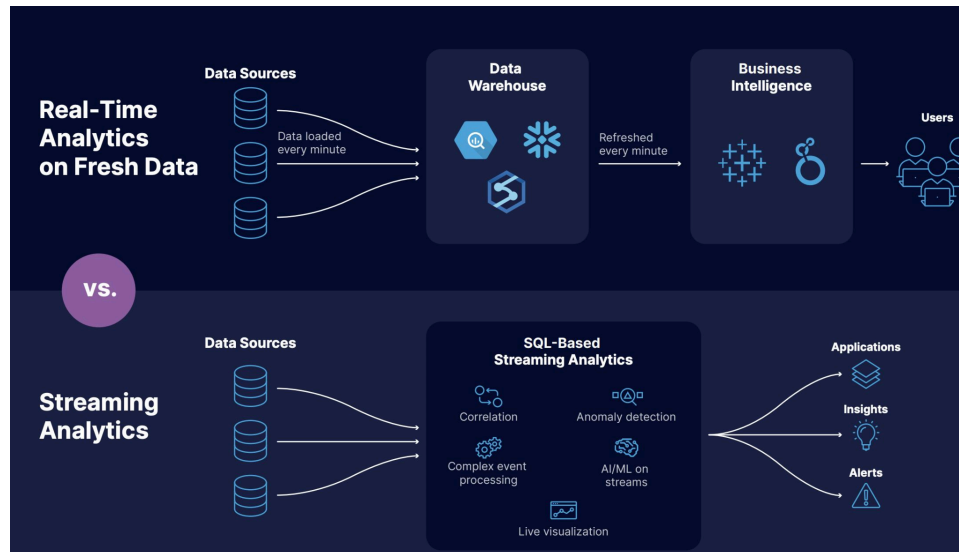


2. Stream Processing

Real-time data streams need to be processed on-the-fly for applications like live analytics, monitoring, and alerting systems.

How Spark can help:

Spark Streaming allows you to process live data streams, dividing the data into batches and processing them using Spark's APIs.

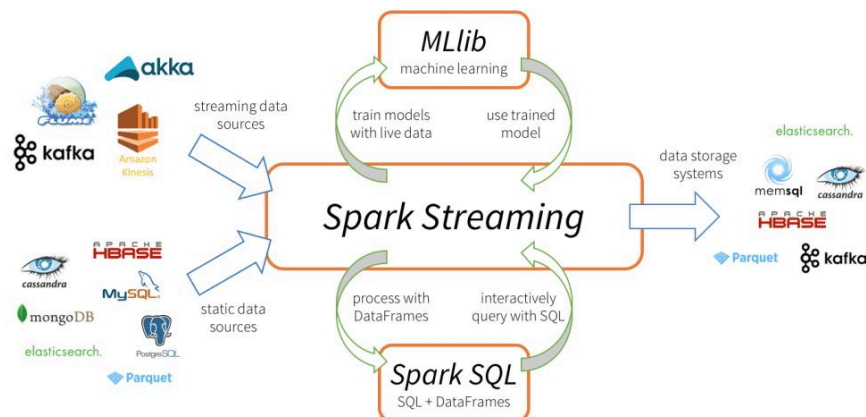


3. Machine Learning and AI

Machine learning and AI applications often require processing large datasets to train and evaluate models.

How Spark can help:

Spark's MLlib library provides scalable machine learning algorithms that can be easily integrated into your data pipeline. It supports various algorithms for classification, regression, clustering, and collaborative filtering.



4. Data Analytics

Enterprises need to analyze large datasets to derive business insights and make data-driven decisions.

How Spark can help:

Spark SQL provides an interface for structured data processing using SQL queries, making it easy to perform data analytics.

5. Log Processing

Analyzing logs is essential for monitoring, troubleshooting, and improving system performance.

How Spark can help:

Spark's ability to process large volumes of data quickly makes it suitable for log processing tasks.

6. Fog Computing

Fog computing involves processing data closer to where it is generated, reducing latency and bandwidth usage.

How Spark can help:

Spark's architecture allows it to be deployed in a fog computing environment, processing data

7. Recommendation Systems

Recommendation systems are widely used in e-commerce and content platforms to provide personalized recommendations to users.

How Spark can help:

Spark's MLlib includes collaborative filtering algorithms, such as ALS (alternating least squares), which can be used to build recommendation systems.

8. Real-time Advertising

Real-time advertising requires processing user interactions in real-time to serve personalized ads.

How Spark can help:

Spark Streaming can process user interaction data in real-time, enabling dynamic ad placement based on user behavior.